MAD report

402411110

Pantry manager

https://github.com/gomotsegangmaluleka393/mad700

Contents

Introduction 3
System design 4
Screenshots 6
Key code snippets 9
Challenges 9
Conclusion 11
References 12

Introduction

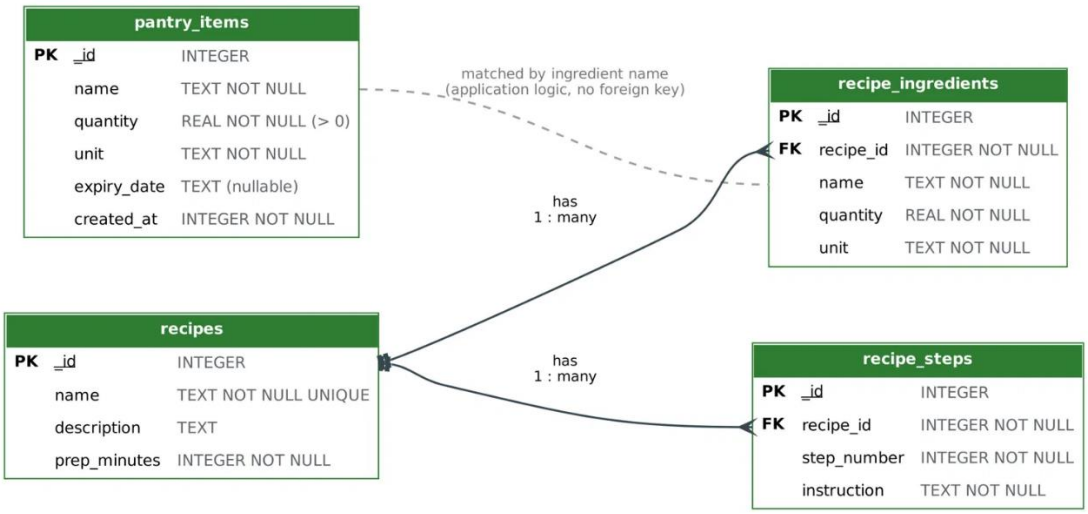
A large share of the food bought by households is thrown away, and much of it is still edible when it is discarded. One common reason is simple: people forget what they already have at home. Ingredients are pushed to the back of the fridge or cupboard, pass their expiry dates unnoticed, and are replaced by new groceries. At the same time, many people end up buying more food because they cannot think of a meal they could make from the odds and ends they already have.

Most recipe apps make this worse rather than better. They are built around inspiration: the user picks a dish and the app produces a shopping list. Even apps that let users search by ingredient usually return recipes that use some of those ingredients, so the user still finds out halfway through that one or two key items are missing, and the result is another trip to the shop.

Pantry Manager is an Android application, written in Java, that helps users reduce food waste by working from what they already own. The user keeps a digital record of their pantry: each ingredient with its quantity, unit and, optionally, its expiry date. The app then compares this pantry against a built-in collection of twenty recipes and suggests only the recipes the user can cook immediately.

The core of the app is its strict-matching rule. A recipe is suggested only if every ingredient it needs is present in the pantry in at least the required quantity. If a recipe needs five ingredients and the user has four, it is not suggested. This guarantees that any recommended recipe can be made without a shopping trip. Because people do not record food in a perfectly consistent way, the matching is designed to cope with everyday differences in wording and measurement.

System design



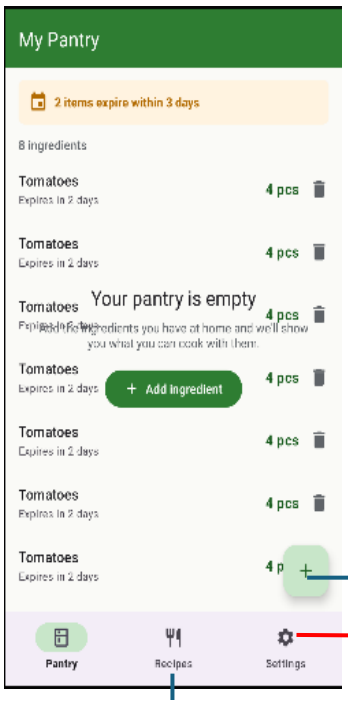


Figure 1.screen

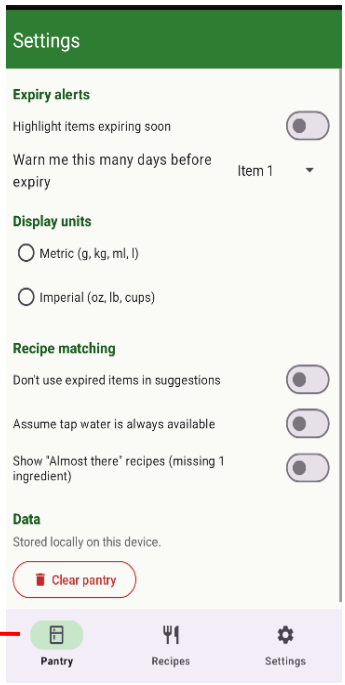


Figure 2. settings screen

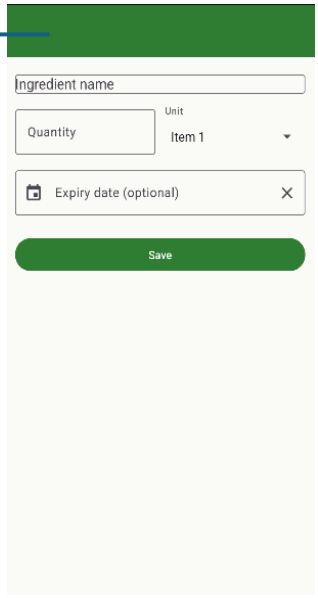


Figure 3 adding/editing ingredient screen

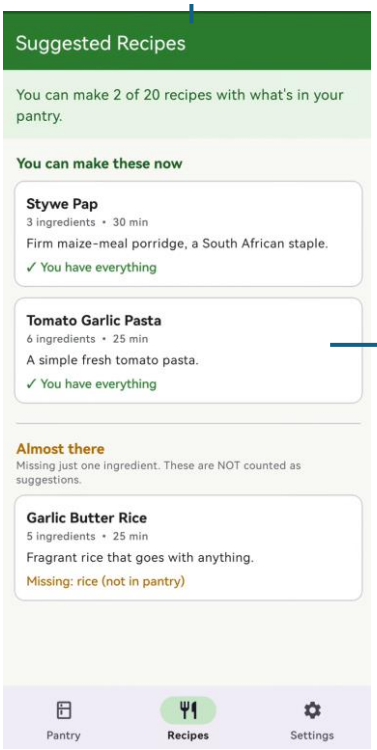


Figure 4. suggested recipes screen

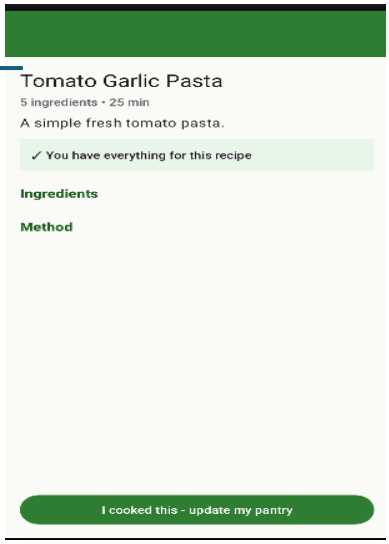


Figure 5.recipe screen

Screenshots

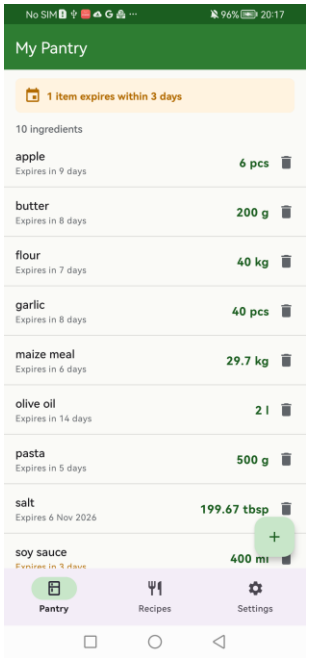


Figure 6.initial pantry

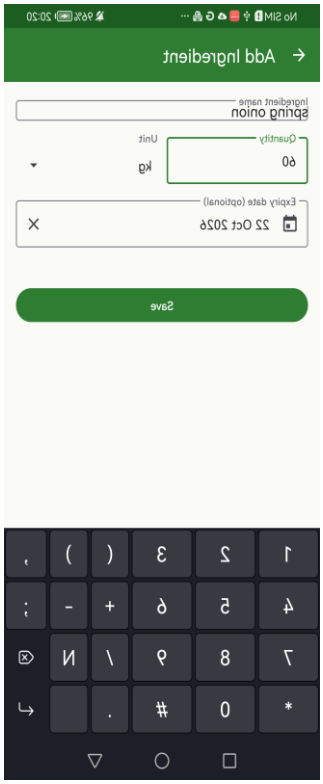


Figure 7. adds the ingredient

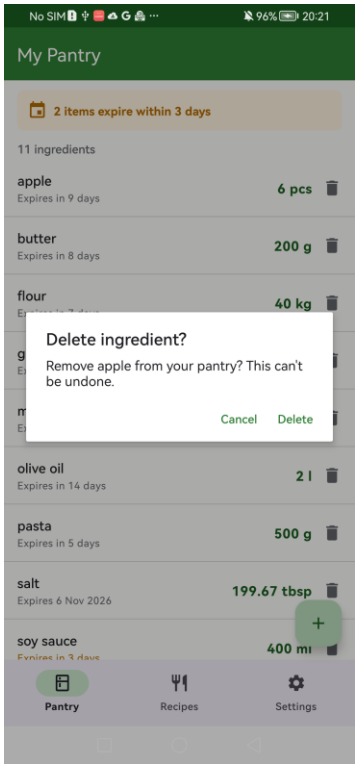


Figure 8. deletes the ingredient

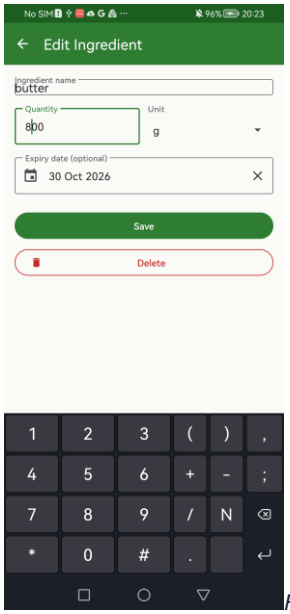


Figure 9. edits ingredients

Figure 10. updates the amount of pantry items when the cook button is clicked

Figure 11. the pantry after all of the pantry operations

Key code snippets

```
private void showDatePicker() {
    LocalDate initial = ExpiryUtils.parse(selectedExpiry);
    if (initial == null) {
        initial = LocalDate.now().plusDays(7);
    }
    DatePickerDialog dialog = new DatePickerDialog(this,
            (view, year, month, day) -> setExpiry(ExpiryUtils.toIso(year,
month, day)),
            initial.getYear(), initial.getMonthValue() - 1,
initial.getDayOfMonth());
    dialog.show();
}
```

displays a calender that the user uses to choose the data of expiry

```
private void confirmClear() {
    new AlertDialog.Builder(this)
            .setTitle(R.string.clear_title)
            .setMessage(R.string.clear_message)
            .setPositiveButton(R.string.clear_confirm, (d, w) -> {
                int removed = db.clearPantry();
                Toast.makeText(this,
getResources().getQuantityString(R.plurals.cleared_count, removed,
removed),
                        Toast.LENGTH_SHORT).show();
                refreshAbout();
            })
            .setNegativeButton(R.string.cancel, null)
            .show();
}
```

displays a message to confirm the data deletion of the pantry, it was written this way to display the alert

```
public PantryItem getPantryItem(long id) {
    try (Cursor c = getReadableDatabase().query(T_Pantry, null, P_ID + " =
?",
            new String[]{String.valueOf(id)}, null, null, null)) {
        return c.moveToFirst() ? fromCursor(c) : null;
    }
}
```

Stores the read data as a Pantry Item, a cursor was needed because it is used for reading data from the database

Challenges

- My original plan was to build the app as a single host Activity, with each screen (Pantry List, Suggested Recipes and Settings) as a Fragment that the bottom

navigation bar would swap in and out. In practice I struggled to get Fragments working reliably. Managing Fragment transactions and the back stack was confusing, the Fragment lifecycle is more complicated than the Activity lifecycle (a Fragment's view can be destroyed and recreated while the Fragment itself still exists), and passing data between Fragments, such as sending the selected ingredient to the edit screen, needed extra code that I found hard to get right. Small mistakes led to screens overlapping, blank screens after rotating the device, or the wrong tab being highlighted in the navigation bar.

Since the brief accepts either Fragments or separate Activities, I changed approach and built each screen as its own Activity. Navigation between screens uses explicit Intents, and data is passed through Intent extras: the Pantry List sends the selected item's database ID to the Add/Edit screen, and the Suggested Recipes screen sends the recipe ID to the Recipe Detail screen. To keep the bottom navigation bar consistent, I put its behaviour in a shared base class (BaseNavActivity) that the three main screens extend, so each screen only has to say which tab it belongs to. Within each screen, the dynamic content is displayed with a RecyclerView instead of being split into Fragments. This made the structure easier to understand and debug, and it uses concepts covered in the module.

- After moving away from Fragments, the RecyclerView became the centre of the app: it displays the pantry items and both recipe lists. Getting it working was a challenge of its own. A RecyclerView needs several pieces to work together (a LayoutManager, a custom Adapter and a ViewHolder), and at first it was not clear to me how they fit together. Typical problems were the list not appearing at all, the list not updating after adding or editing an ingredient until the app was restarted, and working out how to respond when the user taps a row or its delete button, because the click happens inside the Adapter while the database and navigation code live in the Activity.
- How I resolved it
- I worked through the pattern one piece at a time:
    - LayoutManager: each RecyclerView is given a LinearLayoutManager, without which nothing is drawn.
    - Custom Adapter and ViewHolder: I wrote two adapters, PantryAdapter and RecipeAdapter. The ViewHolder looks up the views in a row layout once, and onBindViewHolder fills them with the data for each position.
    - Refreshing the data: the screens reload the data from the SQLite database in onResume(), which runs every time the user returns to the screen, and pass the new list to the adapter, which calls

notifyDataSetChanged(). Changes made on the Add/Edit screen now appear as soon as the user goes back.

- Handling clicks: each adapter defines a small Listener interface that the Activity implements. The adapter reports which item was tapped, and the Activity decides what to do: open the edit screen, confirm a delete, or open the recipe details.
- Empty lists: when the list has no data, the RecyclerView is hidden and a message is shown instead, so the user never sees a blank screen.

Conclusion

Pantry Manager meets its main goal: it lets users record the ingredients they have at home and suggests only the recipes they can cook straight away, which helps to reduce food waste. The app supports adding, viewing, editing and deleting pantry items, stores its data locally in an SQLite database so that it remains after the app is closed, and applies the strict-matching rule so that a recipe is never suggested unless every ingredient is available in the required quantity.

What I learned that the most important thing I learned in this project is how to implement lists in an Android app using RecyclerView. I now understand the roles of the LayoutManager, the Adapter and the ViewHolder, and why RecyclerView reuses row views instead of creating a new view for every item, which keeps long lists fast. I also learned how to connect a list to a database: reading the rows from SQLite, binding them to the list, refreshing it when the data changes, and handling taps on individual rows through a listener interface.

Working through the problem with Fragments also taught me to choose an approach I can understand and debug rather than the one that seems most advanced. Switching to separate Activities connected by Intents gave me a working, reliable app and a better understanding of the Activity lifecycle and how data is passed between screens.

Given more time, the main thing I would improve is the user interface design. The current interface is clear and functional, but quite plain. Improvements I would make include:

•A more polished visual style, with a consistent colour scheme, better spacing and typography, and a clearer visual hierarchy on each screen.

•Images for recipes so that the suggested recipes list is more attractive and easier to scan.

•Friendlier interactions, such as swipe-to-delete with an undo option, and smooth animations when items are added or removed.

•Dark mode and better support for different screen sizes, including tablets and landscape orientation.

•Accessibility improvements, such as larger touch targets, stronger colour contrast and full screen-reader descriptions.

References

Code With Cal (2021) Save Settings in Shared Preferences Android Studio tutorial. https://www.youtube.com/watch?v=-u63b5X2NqE.

Zeeshan Academy (2021) How to use fragments in Android Studio | Understanding Fragments for Multi Layout App. https://www.youtube.com/watch?v=PiExmkR3aps.

Practical Coding (2021) RecyclerView | Everything You need to know. https://www.youtube.com/watch?v=Mc0XT58A1Z4.

Android Knowledge (2022) How to Create ListView in Android Studio using Java - Beginners | Android Knowledge. https://www.youtube.com/watch?v=qfIc_pPg_t4.